🔹 What is UPDATE in MySQL?

UPDATE is a **DML (Data Manipulation Language)** command used to **modify existing records** in a table.

👉 In simple words:

It changes already stored data.

🔹 What is SET?

SET is a **keyword used inside UPDATE** that tells MySQL:

👉 **which column to change and what new value to assign**

🔹 Basic Syntax

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

🔹 Important Points

- UPDATE → selects the table and rows to modify
- SET → defines new values
- WHERE → specifies **which rows to update**

⚠️ If you don't use WHERE, **all rows will be updated** (very dangerous!)

🔹 Example Table

Let's say we have a table:

employees

id	name	salary
1	Ram	30000
2	Sita	35000
3	John	40000

🔹 **Example 1: Update One Column**

UPDATE employees

SET salary = 45000

WHERE id = 1;

👉 Meaning:

- Find employee with id = 1
- Change salary to 45000

🔹 **Example 2: Update Multiple Columns**

UPDATE employees

SET name = 'Ramesh', salary = 50000

WHERE id = 1;

👉 Here:

- SET updates multiple columns at once

🔹 **Example 3: Update Without WHERE (⚠️ Dangerous)**

UPDATE employees

SET salary = 10000;

👉 This will:

- Change salary of **ALL employees**

🔹 **Example 4: Update Using Condition**

UPDATE employees

SET salary = salary + 5000

WHERE salary < 40000;

👉 Meaning:

- Increase salary by 5000 for employees earning less than 40000

🔹 **Real-Time Use Case (Data Engineering)**

Imagine:

- You loaded data into a table (ETL process)
- Later you found incorrect values

👉 You use UPDATE to:

- Fix wrong data
- Normalize values
- Apply transformations

Example:

UPDATE orders

SET status = 'completed'

WHERE payment_received = 'yes';

🔹 **Interview Insights** 💡

Common questions:

👉 Q1: What happens if WHERE is missing?
→ All rows get updated

👉 Q2: Can we update multiple columns?
→ Yes, using comma-separated SET

👉 Q3: Can we use expressions?
→ Yes (like salary = salary + 5000)

👉 Q4: Difference between UPDATE and INSERT?
→

- INSERT → adds new data
- UPDATE → modifies existing data

🔹 **Pro Tip (Very Important)**

Before running UPDATE, always test with:

SELECT * FROM employees WHERE condition;

👉 This ensures you don't accidentally update wrong rows.

🔹 What is DELETE in MySQL?

DELETE is a **DML command** used to **remove existing records (rows)** from a table.

👉 Simple definition:

It deletes data from a table based on a condition.

🔹 Basic Syntax

DELETE FROM table_name

WHERE condition;

🔹 Key Components

- DELETE FROM → specifies the table
- WHERE → decides **which rows to delete**

⚠️ **Without WHERE = deletes ALL rows** (very dangerous)

🔹 Example Table

id	name	salary
1	Ram	30000
2	Sita	35000
3	John	40000

🔹 Example 1: Delete Specific Row

DELETE FROM employees

WHERE id = 2;

👉 Deletes only Sita's record

🔹 Example 2: Delete Using Condition

DELETE FROM employees

WHERE salary < 35000;

👉 Deletes employees with salary less than 35000

🔹 **Example 3: Delete ALL Rows ⚠️**

DELETE FROM employees;

👉 Removes all data, but:

- Table structure remains

🔹 **DELETE vs TRUNCATE (🔥 Interview Favorite)**

Feature	DELETE	TRUNCATE
Type	DML	DDL
WHERE allowed?	✅ Yes	❌ No
Rollback	✅ Possible	❌ Not possible
Speed	Slower	Very fast
Resets AUTO_INCREMENT	❌ No	✅ Yes

👉 Example:

TRUNCATE TABLE employees;

🔹 **Real-Time Use Case (Data Engineering)**

You'll use DELETE when:

- ❌ Removing duplicate records
- ❌ Cleaning invalid data
- ❌ Removing outdated logs

Example:

DELETE FROM logs

WHERE created_at < '2023-01-01';

👉 Deletes old log data (very common in pipelines)

🔹 **Safe Mode Issue (VERY IMPORTANT 🚨)**

You mentioned earlier about safe mode — here's the connection:

If MySQL safe mode is ON, this may fail:

DELETE FROM employees;

👉 Error:

You are using safe update mode...

🔹 **Solution**

Option 1: Use a key column in WHERE

```
DELETE FROM employees
WHERE id > 0;
```

Option 2: Disable safe mode

```
SET SQL_SAFE_UPDATES = 0;
```

🔹 **Pro Tips** 💡

✓ Always run this before DELETE:

```
SELECT * FROM employees WHERE condition;
```

✓ Use transactions in critical systems:

```
START TRANSACTION;
DELETE FROM employees WHERE id = 2;
ROLLBACK; -- or COMMIT
```

🔹 **Interview Insights**

👉 Q: Difference between DELETE and DROP?

- DELETE → removes data
- DROP → removes entire table

👉 Q: Can we recover deleted data?

- Yes (if transaction not committed)

👉 Q: Why is TRUNCATE faster?

- It deallocates memory instead of deleting row-by-row

🔥 **Mentor Advice**

In real companies:

- DELETE = **surgical operation (precise removal)**

- TRUNCATE = **bulk wipe**

👉 One mistake can delete production data — so always be careful.

🔹 **What are Datatypes in MySQL?**

👉 Datatypes define:

What kind of data a column can store.

An attribute that specifies the type of data in a column of our database table.

Examples:

- Numbers → INT
- Text → VARCHAR
- Date → DATE

🔹 **Why Datatypes are Important?**

- Ensure **data accuracy** ✅
- Improve **performance** ⚡
- Reduce **storage usage** 💾
- Avoid **errors in queries** ❌

🔹 **Main Categories of Datatypes**

1️⃣ **Numeric Datatypes (Numbers)**

Used to store integers and decimal values.

🔸 **Integer Types**

Type	Description
TINYINT	Small numbers
SMALLINT	Medium small
INT / INTEGER	Most commonly used
BIGINT	Very large numbers

👉 Example:

```
CREATE TABLE students (
  id INT,
  age TINYINT
);
```

🔸 **Decimal Types (Precise Values)**

Type	Description
FLOAT	Approximate decimal
DOUBLE	More precision than float
DECIMAL(p,s)	Exact values (important for money)

👉 Example:

```
salary DECIMAL(10,2)
```

👉 Meaning:

- Total digits = 10
- Digits after decimal = 2

2️⃣ **String Datatypes (Text)**

Used for storing characters and text.

🔸 **Common Types**

Type	Description
CHAR(n)	Fixed length
VARCHAR(n)	Variable length
TEXT	Large text

🔸 **CHAR vs VARCHAR (Important 🔥)**

Feature	CHAR	VARCHAR
Length	Fixed	Variable

Feature	CHAR	VARCHAR
Speed	Faster	Slightly slower
Storage	Wastes space	Efficient

👉 Example:

name VARCHAR(50)

gender CHAR(1)

3️⃣ Date and Time Datatypes

Used to store date and time values.

Type	Example
DATE	2026-04-07
TIME	14:30:00
DATETIME	2026-04-07 14:30:00
TIMESTAMP	Auto updates time

🔸 **Example**

created_at DATETIME

4️⃣ Boolean Datatype

MySQL does not have a true oolean type.

👉 Instead it uses:

BOOLEAN → TINYINT(1)

- 0 = FALSE
- 1 = TRUE

5️⃣ Special Datatypes

🔸 **ENUM (Very useful 🔥)**

Stores one value from a predefined list.

Status ENUM('pending', 'completed', 'cancelled')

👉 Prevents invalid data entry

🔸 **BLOB (Binary Data)**

Used for:

- Images 🖼️
- Files 📁

🔹 **Real-Time Use Case (Data Engineering)**

Imagine an **orders table**:

```
CREATE TABLE orders (
  order_id INT,
  customer_name VARCHAR(100),
  amount DECIMAL(10,2),
  status ENUM('pending','completed'),
  created_at DATETIME
);
```

👉 Here:

- INT → ID
- VARCHAR → names
- DECIMAL → money
- ENUM → controlled values
- DATETIME → tracking

🔹 **Interview Insights** 💡

👉 Q1: Difference between CHAR and VARCHAR?
→ Fixed vs Variable length

👉 Q2: Why use DECIMAL instead of FLOAT for money?
→ DECIMAL gives exact precision

👉 Q3: What is TIMESTAMP special feature?
→ Auto update on row change

👉 Q4: What is best datatype for large text?
→ TEXT

🔥 Pro Tips (Very Important)

✓ Use VARCHAR instead of CHAR in most cases
✓ Use DECIMAL for financial data
✓ Avoid storing dates as strings ❌
✓ Choose smallest datatype possible (performance boost)

🧠 Mentor Insight

Think like this:

Datatype = **Container for your data**

If you choose wrong container:

- Data gets corrupted
- Performance becomes slow
- Storage gets wasted